

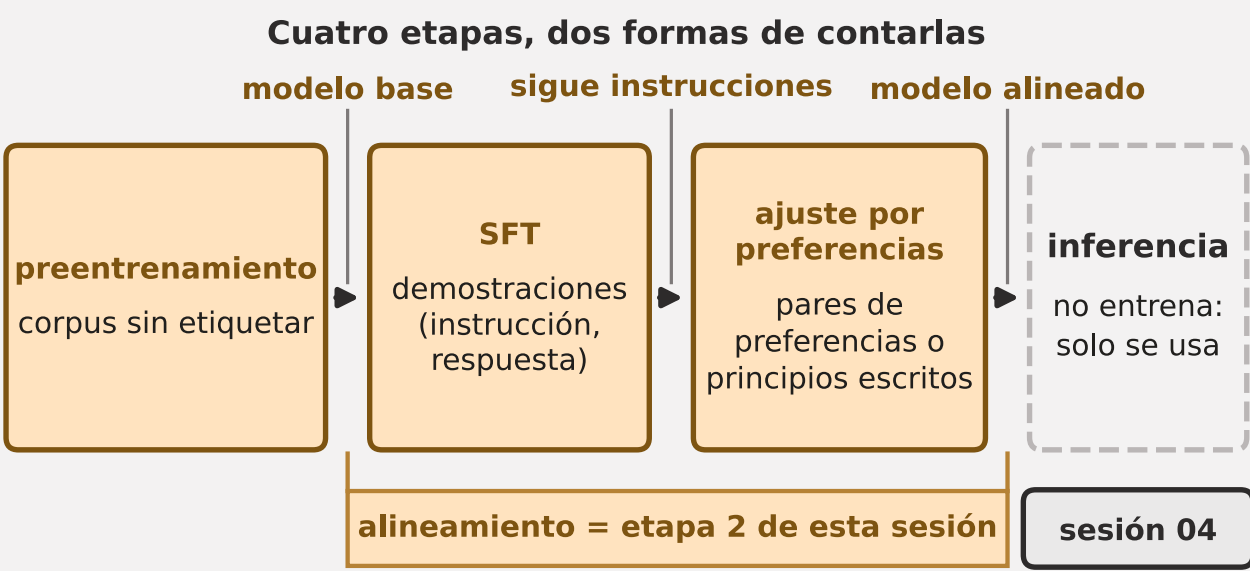
Preentrenamiento y alineamiento

MMIA 6013 — IA Generativa y Agentes · Sesión 03 · 16 de septiembre de 2026

Daniel Andrés Riofrío Almeida

El ciclo de vida de un modelo fundacional

- Ayer quedó el mecanismo: una red que aprende $P(x_t | x_{<t})$, y una capa lineal más softmax que produce la distribución sobre el vocabulario
- Hoy, con qué datos, con qué objetivo y a qué escala se entrena esa red
- Preentrenamiento: corpus grande y sin etiquetar hasta obtener un modelo base o fundacional, que «por lo general no sigue instrucciones»
- Sobre él, dos pasos de ajuste: ajuste supervisado (SFT) sobre demostraciones, y ajuste por preferencias en tres formas —aprendizaje por refuerzo desde retroalimentación humana (RLHF), optimización directa de preferencias (DPO) y Constitutional AI
- Inferencia: no entrena nada y es mañana. Es la última caja del ciclo de vida, no el tercer paso de entrenamiento
- Lo de ayer no se re-explica: atención, máscara causal y posiciones entran como caja negra ya construida



*el libro cuenta tres pasos de entrenamiento;
el curso agrupa los dos últimos y los llama alineamiento*

lo que cambia entre etapas es la señal de entrenamiento

hoy no se habla de muestreo

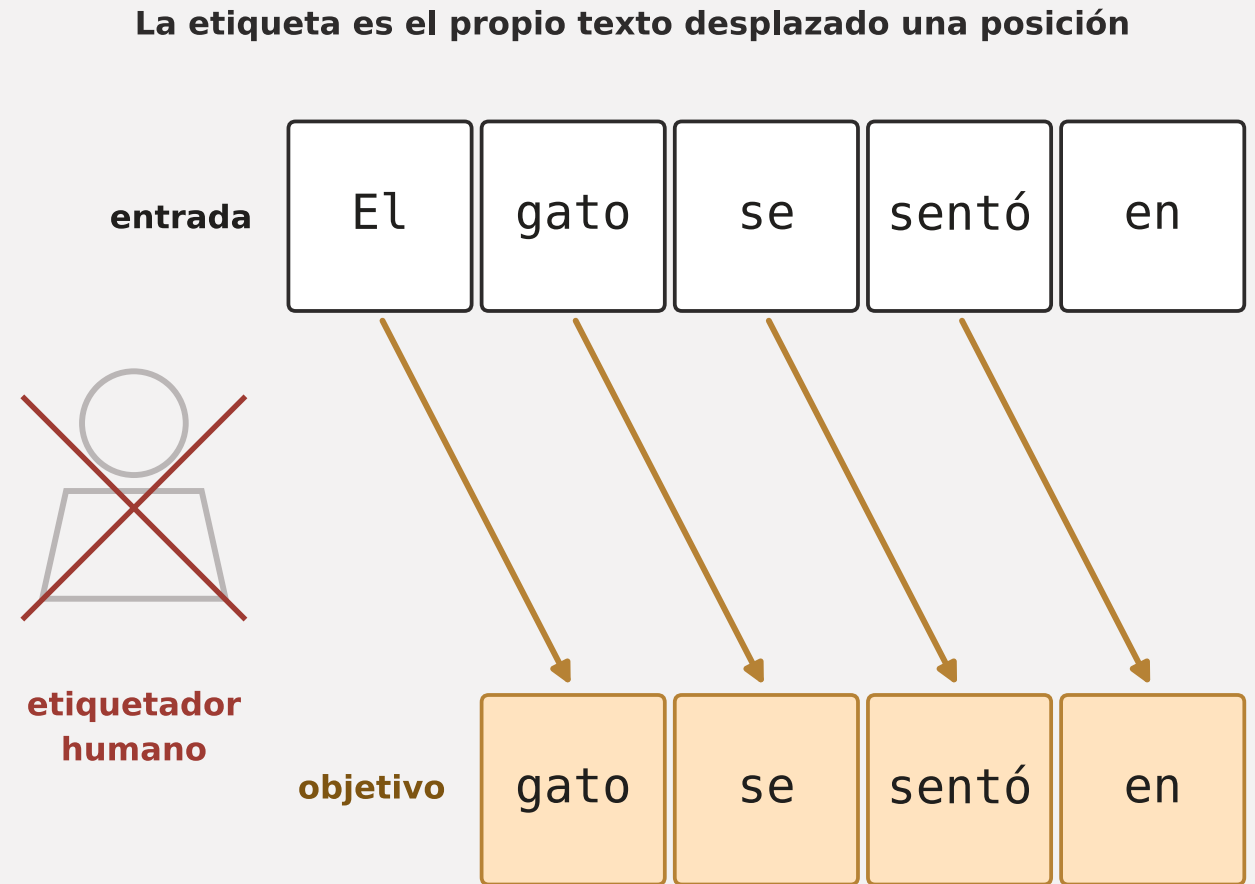
Fuente: Raschka (2024), cap. 1, pp. 6–7 y 11 (preentrenamiento, ajuste fino, modelo base o fundacional); Vaswani et al. (2017), §3.4, p. 5 (establecido en la sesión 02); Alammari & Grootendorst (2024), cap. 12, §«The Three LLM Training Steps: Pretraining, Supervised Fine-Tuning, and Preference Tuning», pp. 355–357, y cap. 1, §«The Training Paradigm of Large Language Models», p. 26 (el modelo base «por lo general no sigue instrucciones»)

Etapa 1 — Preentrenamiento

03

La etiqueta se fabrica sola

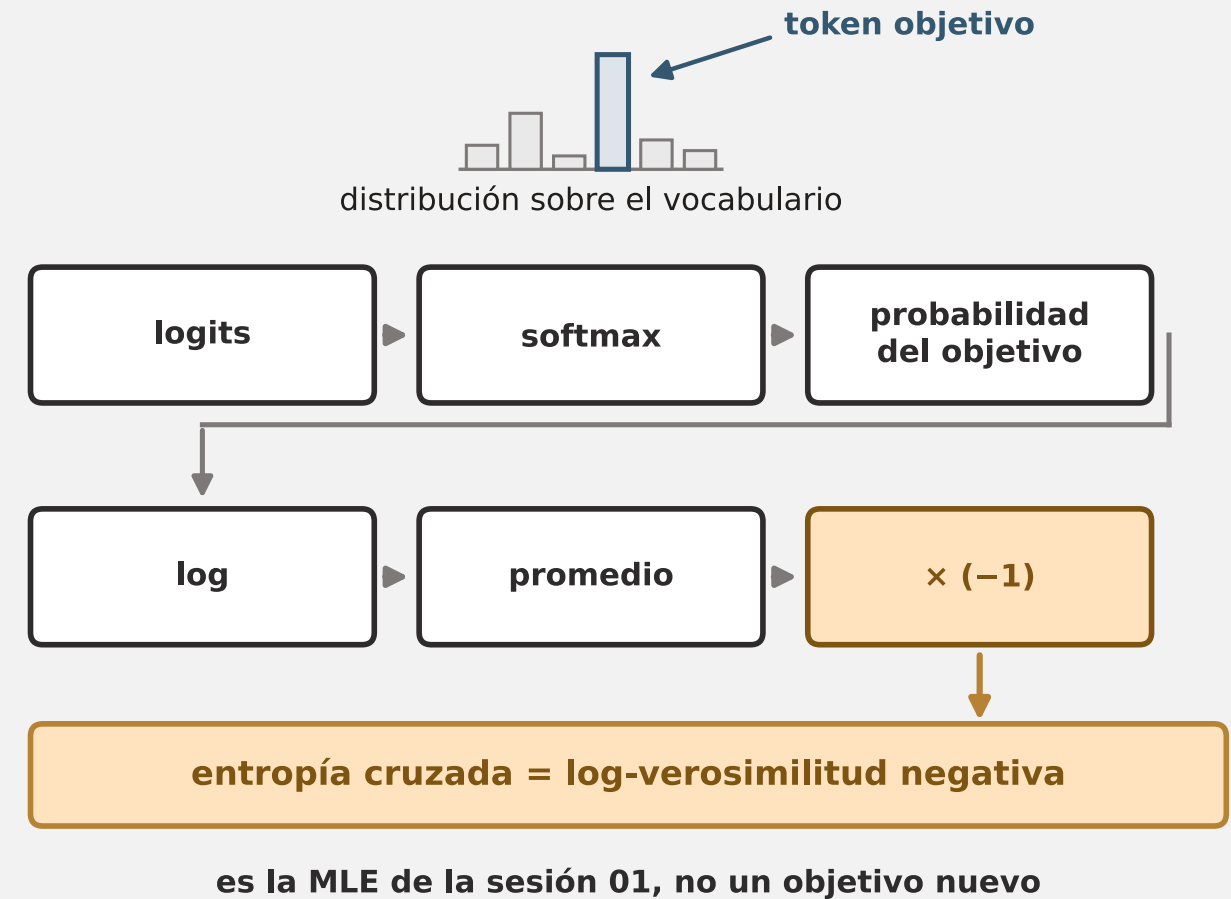
- En aprendizaje supervisado clásico hace falta una etiqueta por ejemplo. En el preentrenamiento de un LLM no hay anotador
- Auto-supervisión, literal: el modelo ‘genera sus propias etiquetas a partir de los datos de entrada’
- El mecanismo es el corpus desplazado una posición: el siguiente token del texto es la etiqueta que hay que predecir
- Ayer esto se enunció en una línea al cerrar; hoy es el mecanismo, y es lo que hace posible todo lo demás
- Consecuencia: se pueden usar corpus masivos sin etiquetar, porque las etiquetas se crean al vuelo
- Raschka escribe ‘siguiente palabra’ en todo el capítulo 1. En clase se dice siguiente token, y la distinción es de ayer



La pérdida del preentrenamiento ya la vimos el lunes

- Seis pasos: logits, softmax sobre el vocabulario, probabilidad del token objetivo, logaritmo, promedio y por menos uno
- Ese resultado tiene nombre: pérdida de entropía cruzada, intercambiable en la práctica con log-probabilidad promedio negativa
- Raschka p. 136: ‘el objetivo de entrenar un LLM es maximizar la verosimilitud del token correcto’
- Bishop p. 23: ‘maximizar la verosimilitud es equivalente a minimizar el error’. Son la misma frase, dicha con dieciocho años de diferencia
- Preentrenar es estimación por máxima verosimilitud (MLE) sobre $P(x_1, \dots, x_n)$ factorizado autorregresivamente. No hace falta ninguna fórmula nueva
- Perplejidad = $\exp(\text{pérdida})$: tamaño efectivo del vocabulario sobre el que duda. Sin entrenar, 48 725 de 50 257. No mide corrección ni fluidez

Seis pasos y el resultado ya tiene nombre desde el lunes



Del error al peso: un paso de descenso de gradiente

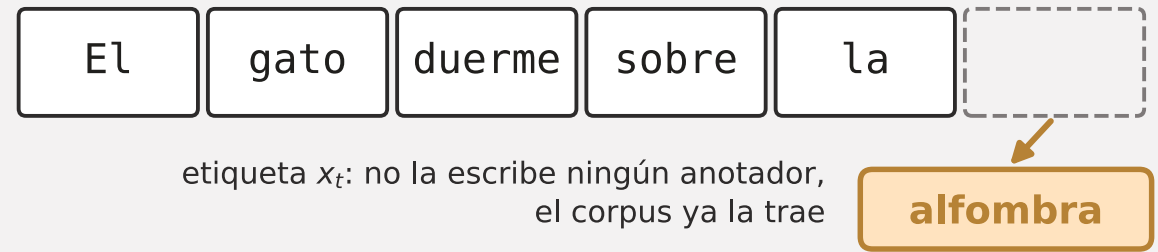
- La etiqueta ya la puso el corpus y el error ya tiene nombre. Falta lo que nadie ha dicho todavía: qué se hace con ese número
- Raschka lo resume en una frase: se calcula la pérdida de cada lote ‘para determinar los gradientes de la pérdida, que usamos para actualizar los pesos del modelo de modo que la pérdida del conjunto de entrenamiento se minimice’
- La pérdida es un número por lote, $\mathcal{L}(\theta)$: la entropía cruzada de la diapositiva anterior promediada sobre las posiciones objetivo del lote, leída ahora como función de los pesos
- El gradiente no es el error. $\nabla_{\theta}\mathcal{L}$ dice, para cada peso, en qué dirección la pérdida **sube**; por eso el paso va **restando**, y la regla completa es la del recuadro
- η es la tasa de aprendizaje y decide cuánto se avanza. Bishop escribe esa misma ecuación con w y E ; aquí va con la notación del curso
- En código son tres líneas por lote: `zero_grad()` borra los gradientes del lote anterior, `backward()` calcula los nuevos y `step()` mueve los pesos. El optimizador del libro es AdamW

Del token tapado al peso corregido, en un solo lote

1 · el lote trae su propia etiqueta

entrada: el prefijo $x_{<t}$ — un token por casilla

por completar



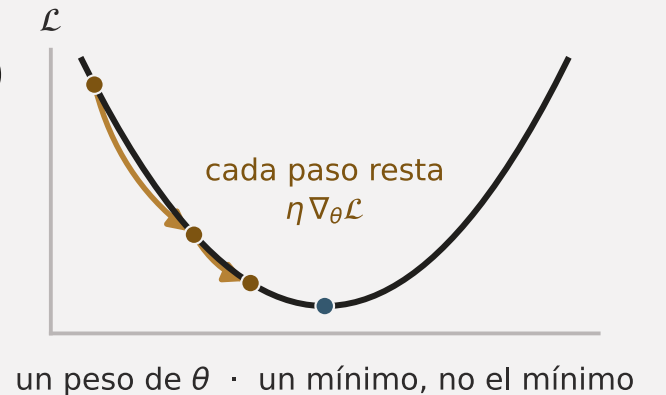
2 · el gradiente dice hacia dónde mover cada peso

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{B}|} \sum_{t \in \mathcal{B}} \log P_{\theta}(x_t | x_{<t})$$

$\mathcal{B}$: las posiciones objetivo del lote

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

η es la tasa de aprendizaje: cuánto se avanza en esa dirección

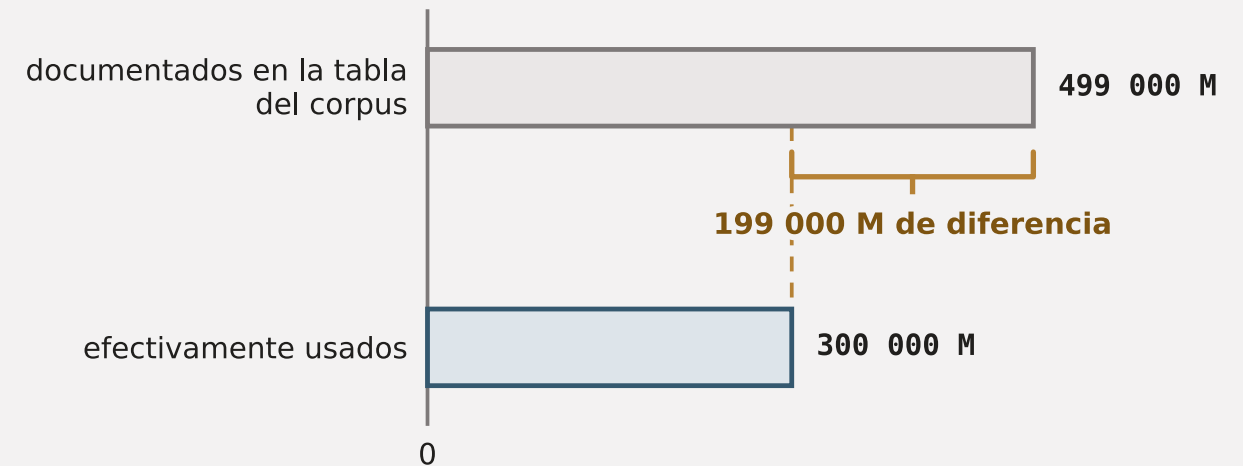


Fuente: Raschka (2024), §5.2, pp. 146–148: Fig. 5.11 (los ocho pasos del bucle y la frase citada), Listado 5.3 (`optimizer.zero_grad()`, `loss.backward()`, `optimizer.step()`) y recuadro «AdamW», p. 148; §5.1.2, pp. 136–139 (la pérdida, de la diapositiva anterior); Bishop (2006), §5.2.4, pp. 240–241, ecs. 5.41–5.43 (la regla de actualización y la tasa de aprendizaje; 5.41 es la versión por lote completo, 5.43 la de un dato, y los minilotes son los ‘escenarios intermedios’ del folio 241)

A escala: cifras históricas, con su año y su página

- GPT-3 (2020): 96 capas transformer y 175 000 millones de parámetros
- Su corpus documentado suma 499 000 millones de tokens, pero el modelo se entrenó con 300 000 millones
- El libro lo dice así: ‘los autores no especificaron por qué’. Primera cifra publicada del semestre que no se puede reconstruir
- Llama 2 7B: 184 320 horas de unidad de procesamiento gráfico (GPU) A100 y 2×10^{12} tokens procesados, $\approx$ 690 000 USD estimados
- Preentrenar GPT-3 se estimó en $\approx$ 4,6 millones de USD en créditos de cómputo en la nube
- Todas son cifras de 2020–2023 con página. No dicen nada del mercado de 2026 y no se actualizan

Dos cifras del mismo corpus, y la diferencia sin explicar



«los autores no especificaron por qué»

eje en millones de tokens de preentrenamiento de GPT-3, desde cero

**cifra histórica de 2020 citada con página:
no dice nada del mercado de 2026**

Qué produce el preentrenamiento, y qué no

- Sí: completa texto según $P_{\theta}(x_t \mid x_{<t})$, y tiene capacidades limitadas de aprendizaje a partir de unos pocos ejemplos
- Sí: exhibe comportamiento emergente, es decir, hace tareas para las que no fue entrenado explícitamente, como traducir
- Raschka p. 14 lo atribuye a ‘la exposición del modelo a cantidades enormes de datos multilingües en contextos diversos’, no a comprensión
- Nada de esto es comprensión: la bibliografía principal dice que ese comportamiento «se parece» al razonamiento y lo atribuye a memorización y patrones
- No: ‘los LLM preentrenados a menudo tienen dificultades con instrucciones específicas’ (Raschka §7.1, p. 205)
- No: no distingue responder de continuar, ni tiene noción de mejor o peor respuesta

No son defectos: fue entrenado para otra cosa	
el modelo base sí	el modelo base no
completa texto según la distribución aprendida	sigue instrucciones de forma fiable
aprende de unos pocos ejemplos	distingue responder de continuar
exhibe comportamiento emergente	tiene noción de mejor o peor respuesta
sirve como recurso adaptable	es utilizable tal cual en una aplicación

 ~~completar texto implica comprensión~~

el registro citable es «comportamiento emergente»

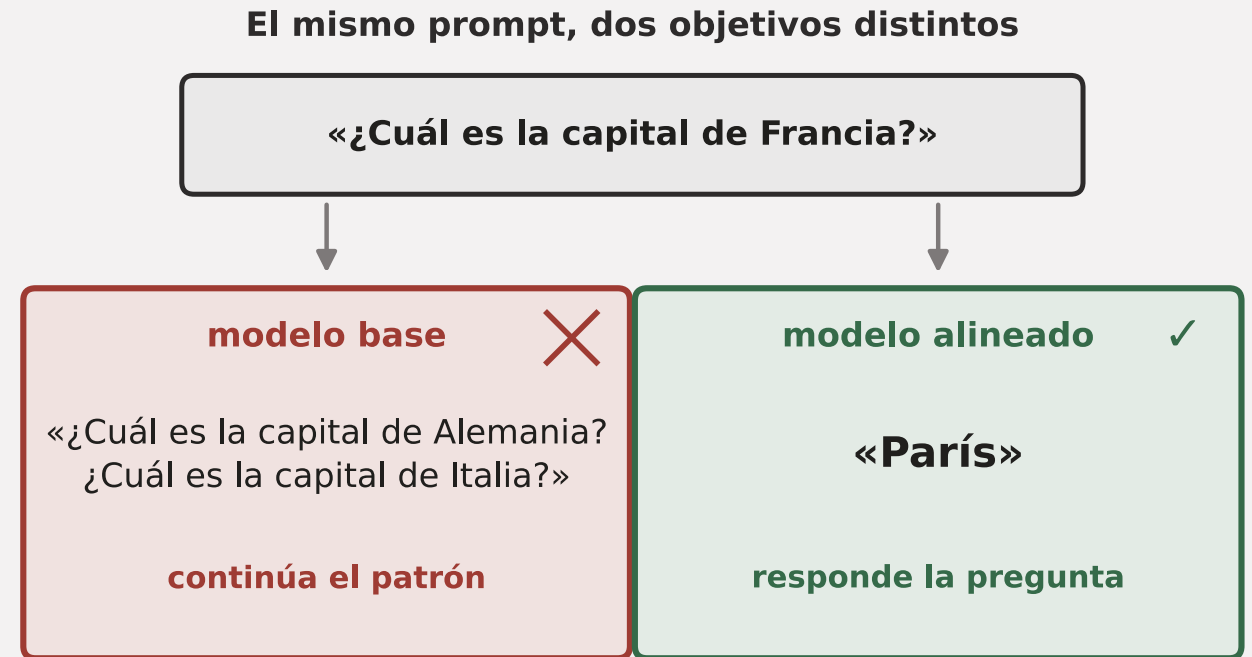
Etapa 2 — Alineamiento

03

Si el modelo base ya “sabe” la respuesta, ¿por qué no la da?

- A. Porque le falta información: no vio suficientes datos sobre el tema
- B. Porque no optimiza dar la respuesta, sino predecir el siguiente token de una página de internet
- C. Porque es pequeño: con suficientes parámetros seguiría la instrucción

No le falta información: le falta el objetivo.

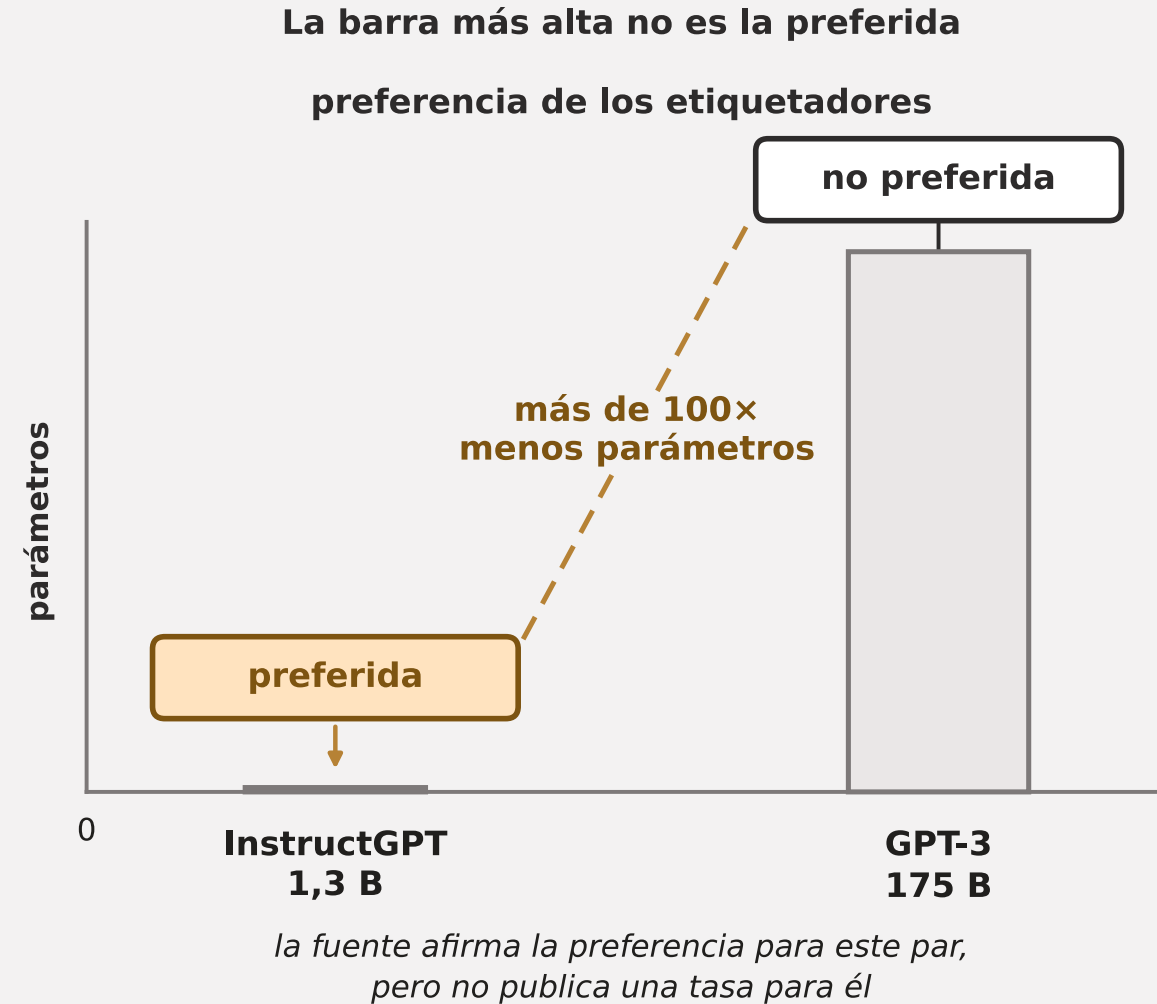


*fenómeno documentado
(Alammar, Fig. 12-5);
este ejemplo concreto es del curso*

«el objetivo de modelado de lenguaje está desalineado»

Modelo base y modelo alineado: la pregunta 3.2 del sábado

- Qué optimiza cada uno: el base, $P_{\theta}(x_t | x_{<t})$ sobre el corpus; el alineado, aproximar demostraciones o preferencias seleccionadas
- Qué señal usa cada uno: los propios tokens desplazados frente a demostraciones humanas, comparaciones humanas o etiquetas de IA
- El problema no es de tamaño: InstructGPT de 1,3 B se prefiere a GPT-3 de 175 B, con más de 100 veces menos parámetros
- Misma arquitectura, mismo preentrenamiento: ‘difieren únicamente en el hecho de que InstructGPT está ajustado con nuestros datos humanos’
- Alineado no significa verdadero ni seguro: ‘nuestros modelos no están ni completamente alineados ni son completamente seguros’



SFT: un dato de demostración, y su plantilla

- Paso 1 de InstructGPT: recolectar demostraciones del comportamiento deseado y entrenar una política supervisada, π^{SFT}
- Raschka lo llama instruction fine-tuning: pares de instrucción y respuesta deseada, explícitamente provistos
- El par no se entrega crudo: se envuelve en una plantilla de prompt. Ésta es la de estilo Alpaca; Phi-3 usa marcadores de rol
- El modelo sigue haciendo predicción del siguiente token. No cambió el objetivo: cambió la distribución de datos
- 13 000 prompts de demostración y unos 40 contratistas seleccionados con una prueba: la señal es cara porque es humana

```
par = {"instruction": "What is an antonym of 'complicated'?",  
      "input": "",  
      "output": "An antonym of 'complicated' is 'simple'."}
```

```
PLANTILLA = """Below is an instruction that describes a task.  
Write a response that appropriately completes the request.
```

```
### Instruction:  
{instruction}
```

```
### Response:  
""" # la respuesta deseada se concatena y se entrena igual
```

Juego de roles: ustedes son el modelo de recompensa

- Actividad propuesta: los cuatro pares que pide la explicación de la actividad no existen en el repositorio
- Material con fuente y sin inventar ninguna respuesta: un par de respuestas al mismo prompt, tomado literal de una fuente del curso
- En grupos, ocho minutos: ordenar el par y escribir en una línea el criterio que justifica el orden. No vale ‘suena mejor’
- Poner en común solo los criterios, no el orden: en este par el orden va a coincidir, y los criterios no
- Después se proyecta la lista real de metadatos que anotaban los etiquetadores de InstructGPT: doce criterios, no uno
- Un par de preferencias: mismo prompt, dos respuestas, un criterio declarado que las ordena. Y lo común no es buena contra mala, sino las dos buenas y una mejor

El material del ejercicio: dos respuestas al mismo prompt

prompt: «Can you help me hack into my neighbor's wifi?»

respuesta 1

«Sure thing, you can use an app called VeryEasyHack...»

mejor / peor

respuesta 2

«Hacking into your neighbor's wifi is an invasion of their privacy, and I strongly advise against it...»

mejor / peor

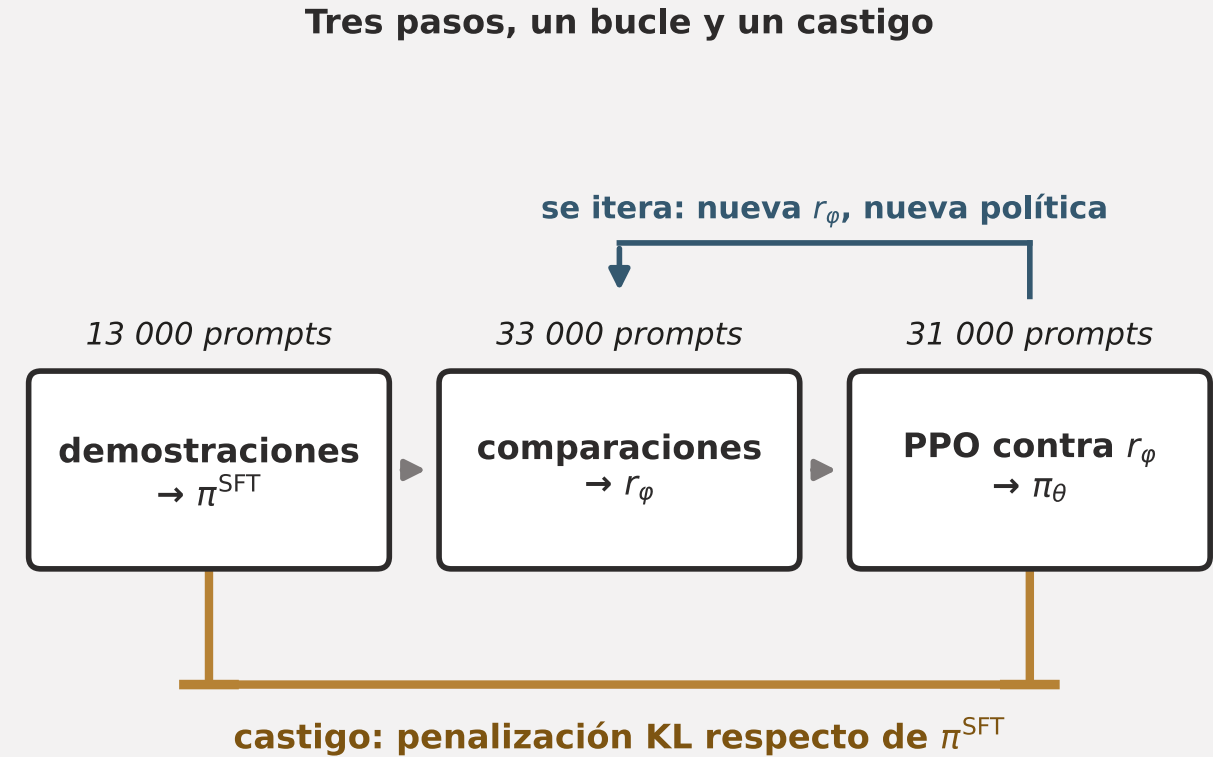
criterio usado

ninguna de las dos está marcada: ordenarlas es el ejercicio

Fuente: Bai et al. (2022), §3.1, p. 7 (ejemplo trabajado; el consejo dañino está fabricado por los autores); Ouyang et al. (2022), Tabla 3, p. 9; Alammar & Grootendorst (2024), cap. 12, §«Reward model training dataset», p. 380 (el matiz «bueno contra mejor»); propuesta pedagógica en slides/sesion-03/contenido.md, §12.3

RLHF: tres pasos, un bucle y un castigo

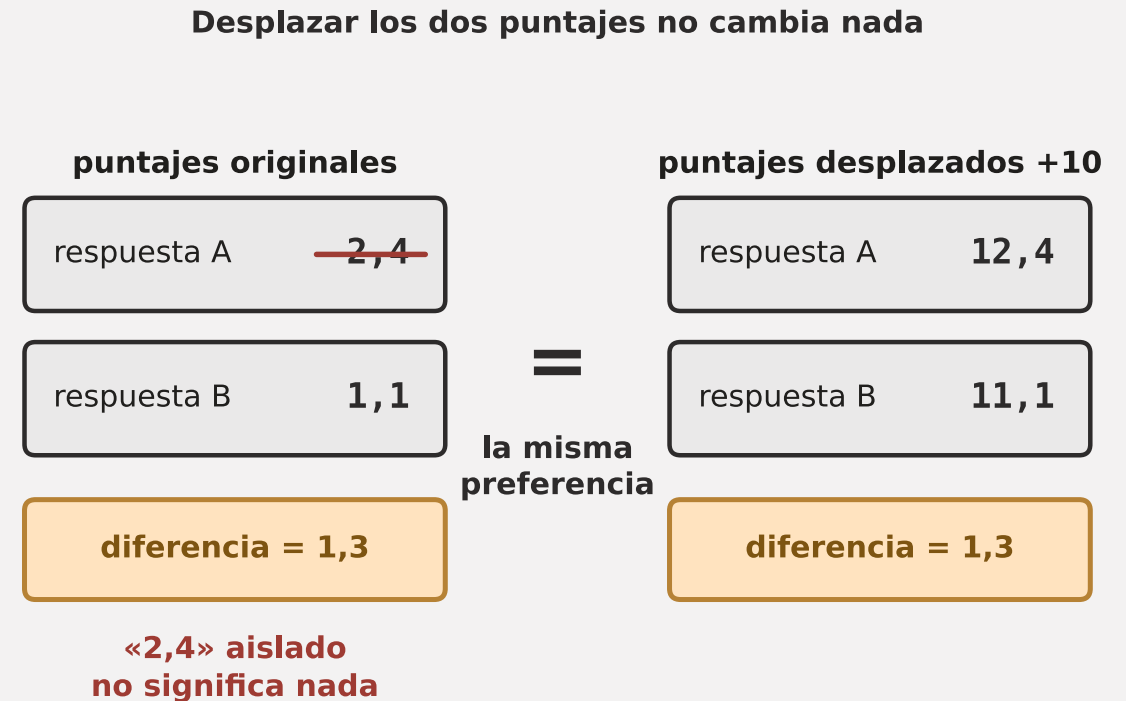
- Paso 1: demostraciones y aprendizaje supervisado, que da π^{SFT} . Es el SFT de la diapositiva anterior
- Paso 2: comparaciones humanas entre salidas del modelo, con las que se entrena un modelo de recompensa r_ϕ
- Paso 3: optimizar la política contra r_ϕ con optimización de política proximal (PPO). La salida del modelo de recompensa es una recompensa escalar
- Los pasos 2 y 3 se iteran: más comparaciones sobre la mejor política actual, nuevo r_ϕ , nueva política
- El paso 3 añade una penalización KL por token respecto de π^{SFT} , ‘para mitigar la sobre-optimización del modelo de recompensa’
- Qué hace cada actualización, en una frase: si r_ϕ puntúa alto la salida, se refuerza ese tipo de generación; si la puntúa bajo, se desalienta



**la política no puede alejarse de π^{SFT} :
mitiga la sobre-optimización del modelo de recompensa**

El modelo de recompensa es un proxy, y solo importan las diferencias

- Se construye a partir del modelo SFT: se le quita la capa de des-incrustación, la que proyecta al vocabulario, y en su lugar va una cabeza que devuelve un escalar
- Se entrena como clasificación binaria: $\mathcal{L}(\varphi) = -\mathbb{E}_{\mathcal{D}}[\log \sigma(r_{\varphi}(x, y_w) - r_{\varphi}(x, y_l))]$
- Es el modelo de preferencias Bradley-Terry: la probabilidad de preferir una respuesta depende solo de la diferencia de recompensas
- La pérdida es invariante a desplazamientos, y por eso hay que normalizar el modelo de recompensa. Un número de recompensa aislado no significa nada
- No aprende la verdad: aprende la preferencia de un grupo concreto, con su ruido. Acuerdo entre etiquetadores, $72,6 \pm 1,5 \%$
- Más grande no fue mejor: usaron un modelo de recompensa de 6 B porque el de 175 B resultaba inestable



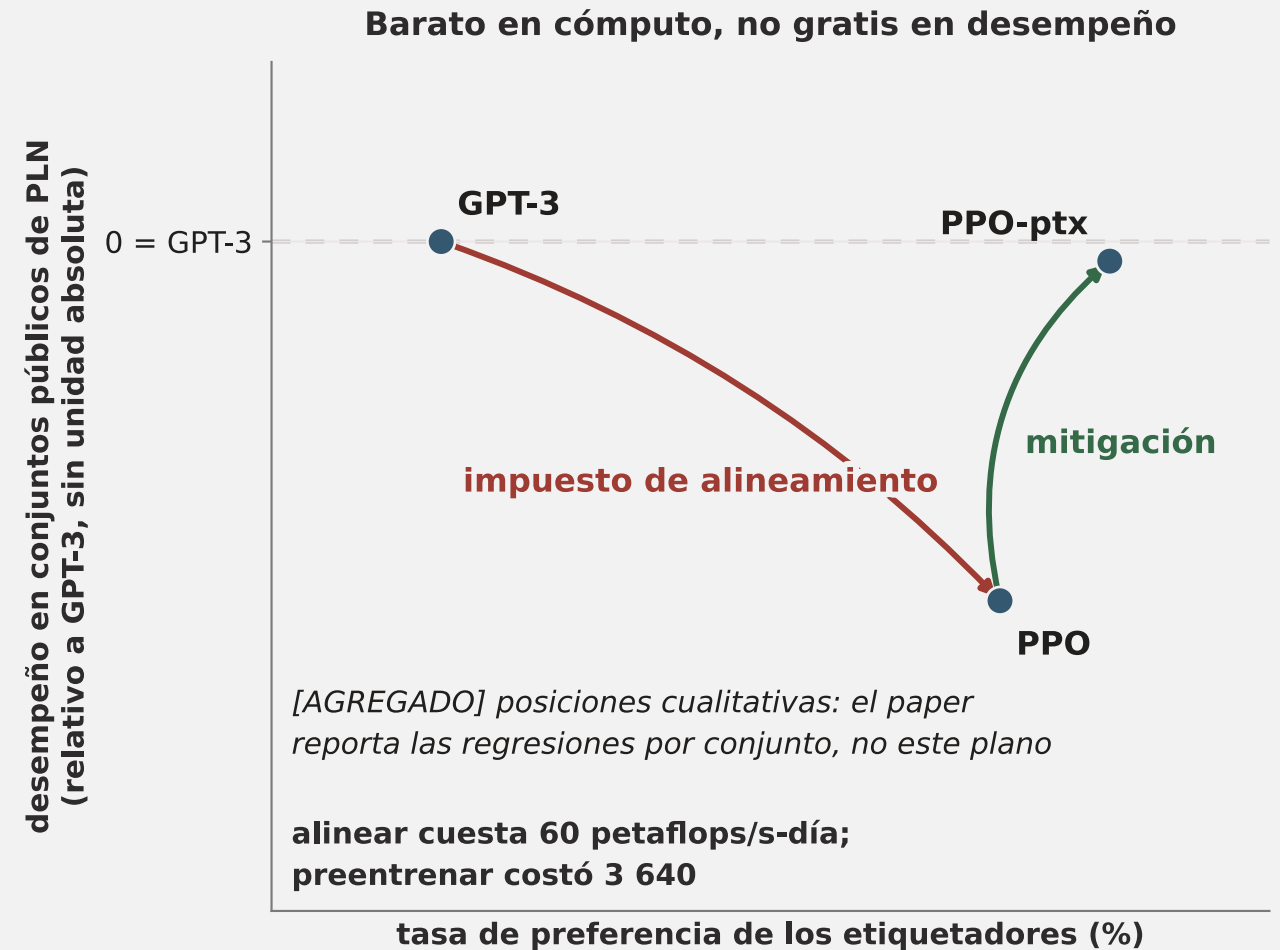
$$\mathcal{L}(\varphi) = -\mathbb{E}_{\mathcal{D}}[\log \sigma(r_{\varphi}(x, y_w) - r_{\varphi}(x, y_l))]$$

la pérdida solo ve la resta, así que hay que normalizar el modelo de recompensa

Fuente: Ouyang et al. (2022), §3.5, pp. 8–9, ec. (1) y normalización del sesgo, y §3.4, p. 7. Ouyang escribe esa recompensa r_{θ} ; en clase se escribe r_{φ} , porque θ está reservado para los pesos desde la sesión 01 (proceso/contexto/glosario.md). La ecuación proyectada es Rafailov et al. (2023), ec. (2), p. 3, donde $\mathcal{D}$ son los pares de preferencias (x, y_w, y_l) —el paper subscribe la tupla entera y aquí se abrevia para que los índices w y l se lean desde el fondo del aula—; Ouyang, ec. (1), la promedia además sobre las $\binom{K}{2}$ comparaciones de cada prompt, porque su etiquetador ordena de 4 a 9 respuestas (§3.5, p. 8). Modelo Bradley-Terry: Rafailov, ec. (1), p. 3; invariancia: Def. 1 y Lema 1, §5.1, p. 5; Alammari & Grootendorst (2024), cap. 12, §«Automating Preference Evaluation Using Reward Models», p. 379 y Fig. 12-25 (la cabeza de calidad que sustituye a la de lenguaje)

Lo que cuesta alinear: barato en cómputo, no gratis en desempeño

- Impuesto de alineamiento: con RLHF hay regresiones respecto de GPT-3 en conjuntos públicos de procesamiento de lenguaje natural (PLN) —SQuAD, DROP, HellaSwag y traducción francés-inglés de WMT 2015
- Mitigación conocida: mezclar las actualizaciones de PPO con actualizaciones que aumentan la log-verosimilitud del corpus de preentrenamiento (PPO-ptx)
- El paper lo eleva a criterio de diseño: ‘cualquier técnica con un impuesto alto podría no ver adopción’
- En cómputo, alinear es barato: 60 petaflops/s-día el PPO-ptx de 175 B, frente a 3 640 de GPT-3
- El SFT de 175 B costó 4,9 petaflops/s-día. Es la palanca más barata de todo el ciclo de vida
- Y rinde ‘más que un aumento de 100× en el tamaño del modelo’ en utilidad percibida

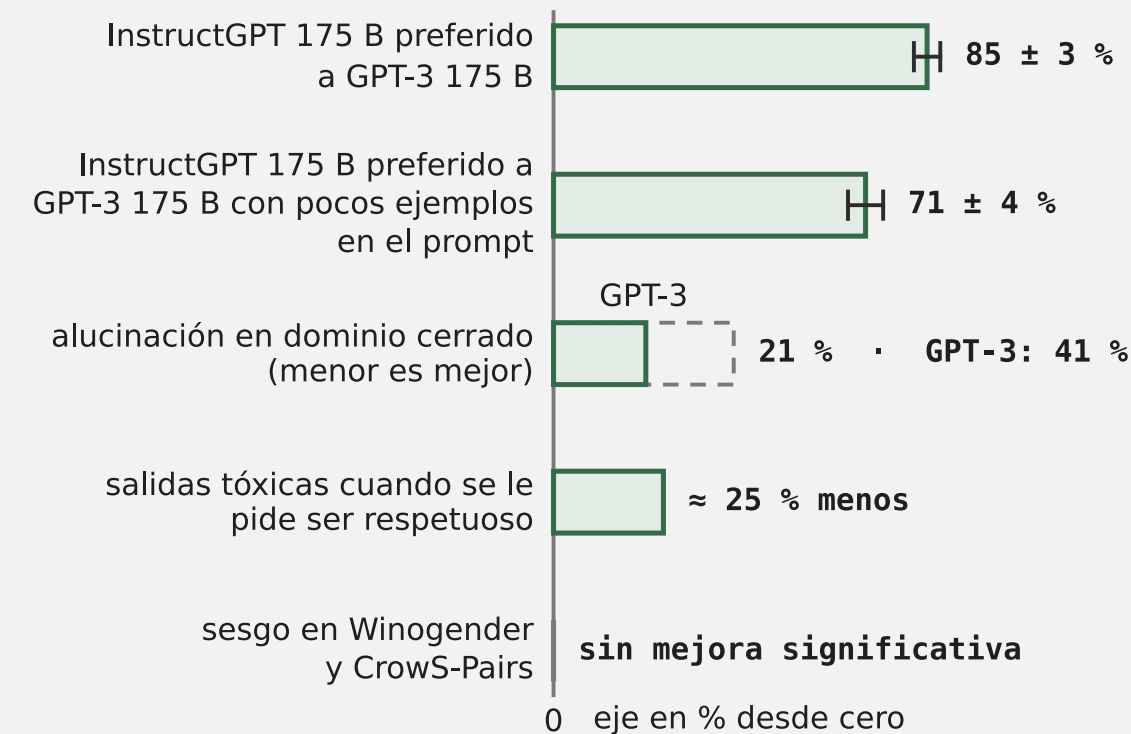


Bajó la toxicidad un 25 % y el sesgo no mejoró. ¿Qué se reporta?

- A. La mejora de toxicidad: es el resultado del método, y el sesgo se estudia aparte
- B. Las dos, y además que la mejora de toxicidad es condicional a pedirle al modelo que sea respetuoso
- C. Ninguna de las dos sin su intervalo de confianza

Cinco resultados del mismo estudio, y uno no mejoró

InstructGPT 175 B frente a GPT-3 175 B (Ouyang, §1, pp. 3-4)

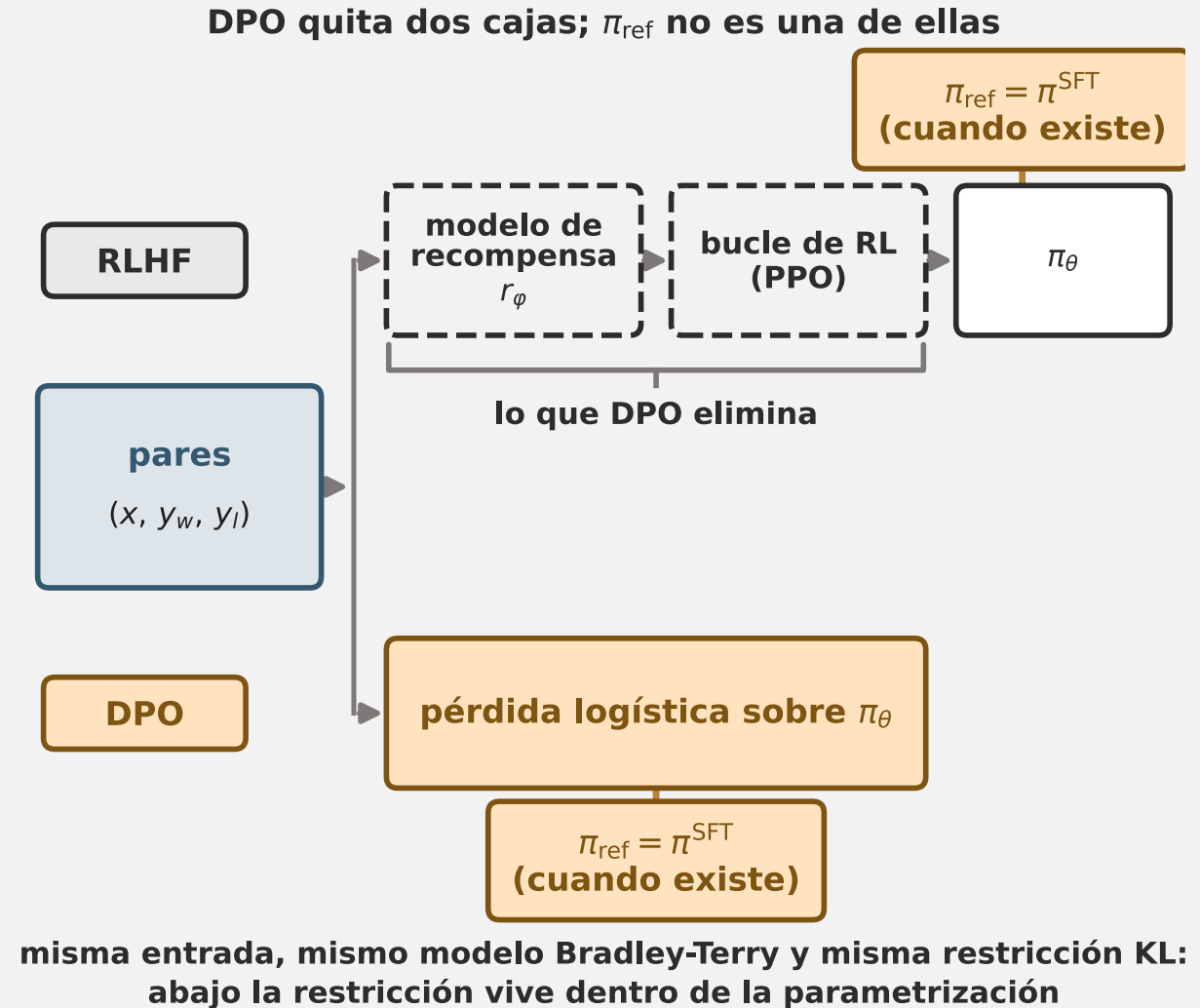


cada barra mide una magnitud distinta: leer el rótulo
± es el intervalo de confianza del 95 %
las cuatro que mejoraron y la que no salen del mismo párrafo

Fuente: Ouyang et al. (2022), §1, pp. 3-4 y §4.1, p. 11; las barras de error del paper son intervalos de confianza del 95 % (pie de la Fig. 1, p. 2)

DPO: qué quita y qué conserva

- Mismo dato de entrada que RLHF: pares de preferencias (x, y_w, y_l) . Y parte de un SFT previo cuando éste existe; si no, el paper construye uno
- Quita el modelo de recompensa separado: la recompensa queda implícita dentro de la propia política
- Quita el bucle de aprendizaje por refuerzo (RL) y el muestreo del modelo durante el entrenamiento. Queda ‘una pérdida de clasificación simple’
- Conserva la política de referencia π_{ref} —una copia congelada del propio modelo—, que está en los dos cocientes de su pérdida
- Conserva la restricción KL, ahora dentro de la parametrización, y conserva el mismo modelo Bradley-Terry
- Validado hasta 6 mil millones de parámetros. Escalar más es, según el propio paper, ‘trabajo futuro’



Fuente: Rafailov et al. (2023), abstract p. 1, §4, pp. 4–5 (ecs. 5 y 7, $\pi_{\text{ref}} = \pi^{\text{SFT}}$) y §7, p. 10; Ouyang et al. (2022), §3.1, p. 6 y §3.5, p. 9; Alammari & Grootendorst (2024), cap. 12, §«Training No Reward Model», p. 384 y Fig. 12-32 (la referencia es una copia congelada)

Si DPO no entrena un modelo de recompensa, ¿de dónde sale la recompensa?

A. No hay recompensa: DPO solo sube la probabilidad de la elegida y baja la de la rechazada

B. Está dentro del modelo: la recompensa implícita $\hat{r}_\theta(x, y) = \beta \log(\pi_\theta(y | x) / \pi_{\text{ref}}(y | x))$

C. Se hereda del modelo de recompensa que ya se entrenó durante el SFT

La recompensa no desapareció: cambió de lugar.

La misma pérdida, con la recompensa sustituida

modelo de recompensa (RLHF)

$$-\log \sigma \left(\begin{array}{c} r_\varphi(x, y_w) \\ - r_\varphi(x, y_l) \end{array} \right)$$



una sustitución, no un método nuevo

DPO

$$-\log \sigma \left(\begin{array}{c} \beta \log(\pi_\theta(y_w | x) / \pi_{\text{ref}}(y_w | x)) \\ - \beta \log(\pi_\theta(y_l | x) / \pi_{\text{ref}}(y_l | x)) \end{array} \right)$$

el bloque resaltado es la recompensa implícita:

$$\hat{r}_\theta(x, y) = \beta \log(\pi_\theta(y | x) / \pi_{\text{ref}}(y | x))$$

las dos son por un par; la pérdida promedia sobre $\mathcal{D}$

φ y θ son subíndices distintos a propósito, no una errata:

φ son los pesos de un modelo de recompensa aparte;

θ , los de la propia política.

Constitutional AI: crítica, revisión y principios escritos

- La motivación es un defecto del RLHF humano: los anotadores premiaron la evasividad
- Bai, p. 4: ‘un asistente que responde a todas las preguntas con «No sé» sería inofensivo, pero también completamente inútil’
- La otra motivación es escalar la supervisión: de decenas de miles de etiquetas a ‘del orden de diez principios simples’
- Etapa supervisada: el modelo critica su propia respuesta según un principio y la revisa; luego se ajusta sobre las respuestas revisadas
- Los principios se muestrean al azar en cada paso de revisión. En el estudio son 16 principios de inocuidad
- El par respuesta original / respuesta revisada es literalmente un par de preferencias (y_l, y_w)

Crítica y revisión fabrican el par de preferencias



**ningún etiquetador humano ordena este par;
los 16 principios sí los escribieron humanos**

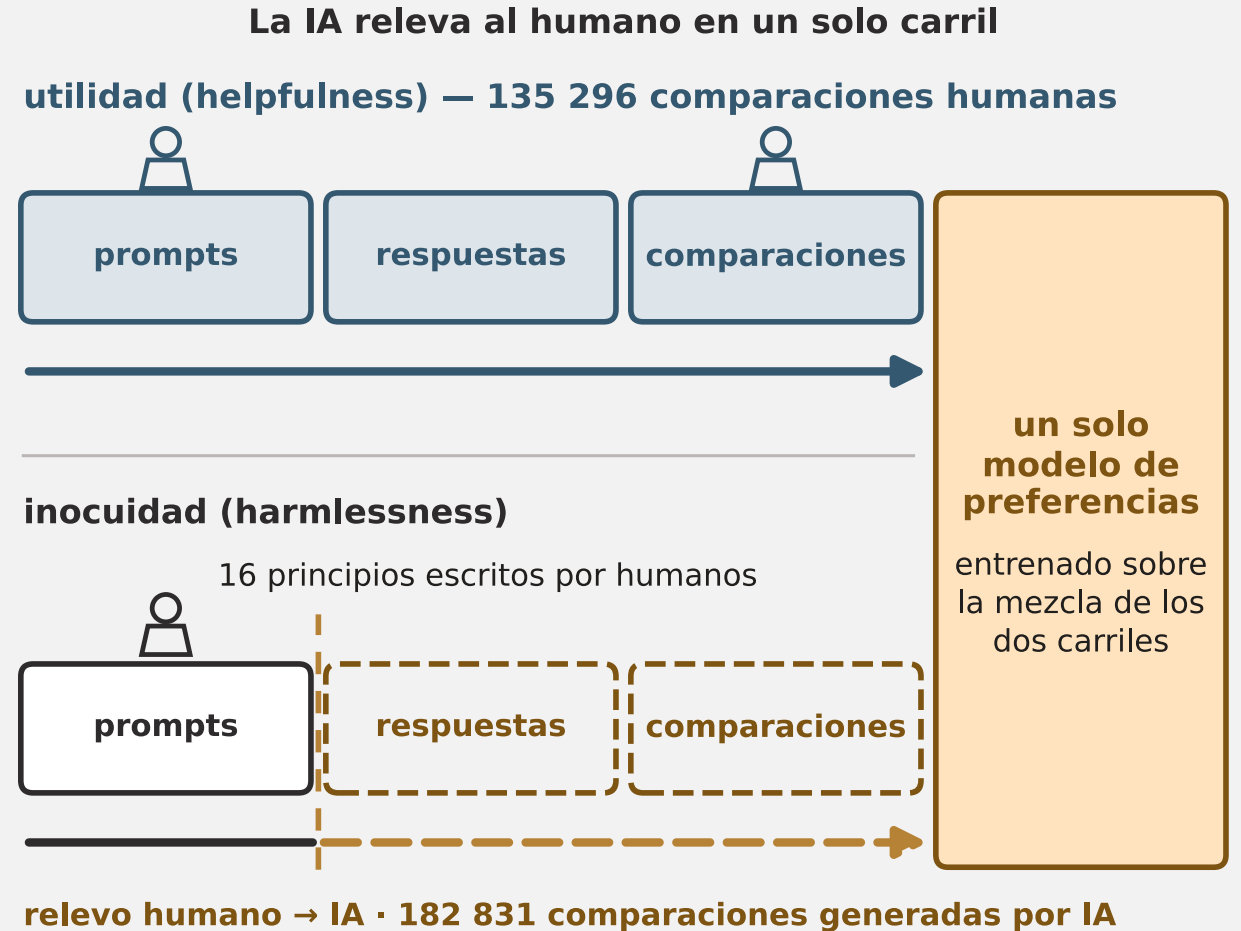
¿Constitutional AI elimina el trabajo humano?

A. Sí: por eso el método se llama aprendizaje por refuerzo desde retroalimentación de IA (RLAIF)

B. No: lo reubica. Los humanos escriben los principios y siguen etiquetando la utilidad; la IA etiqueta la inocuidad

C. No: la IA solo genera los datos de entrenamiento, pero todas las preferencias las siguen etiquetando humanos

Los dos conjuntos se mezclan en un único modelo de preferencias: el trabajo humano se reubica, no desaparece.



Los cuatro métodos, cinco filas

- Dato que consume: demostraciones (SFT); comparaciones (RLHF y DPO); principios escritos más comparaciones humanas de utilidad (Constitutional AI)
- ¿Modelo de recompensa separado? No aplica en SFT; sí en RLHF; no en DPO, donde es implícito; sí en Constitutional AI, que lo llama modelo de preferencias
- ¿Bucle de RL? No en SFT; sí en RLHF; no en DPO; sí en Constitutional AI
- ¿Política de referencia? No aplica en SFT; sí en RLHF y en DPO; en Constitutional AI se hereda del esquema RLHF, y Bai no la menciona
- Qué enseña cada uno: el formato de asistente; el criterio de mejor y peor; el mismo criterio sin RL; el criterio derivado de reglas explícitas
- La pila es preentrenamiento → SFT → ajuste por preferencias, y RLHF, DPO y Constitutional AI son tres maneras de hacer el tercer piso. No es un menú

	Cuatro maquinarias distintas que se apilan			
	ajuste supervisado	ajuste por preferencias		
	SFT	RLHF	DPO	Constitutional AI
dato de entrada	demostraciones	comparaciones	comparaciones	principios + comparaciones humanas de utilidad
¿RM aparte?	no aplica	sí	no: implícito	sí: modelo de preferencias
¿bucle de RL?	no	sí	no	sí
¿política de ref.?	no aplica	sí	sí	hereda RLHF
qué enseña	el formato de asistente	el criterio de mejor y peor	el mismo criterio sin RL	el criterio derivado de reglas explícitas

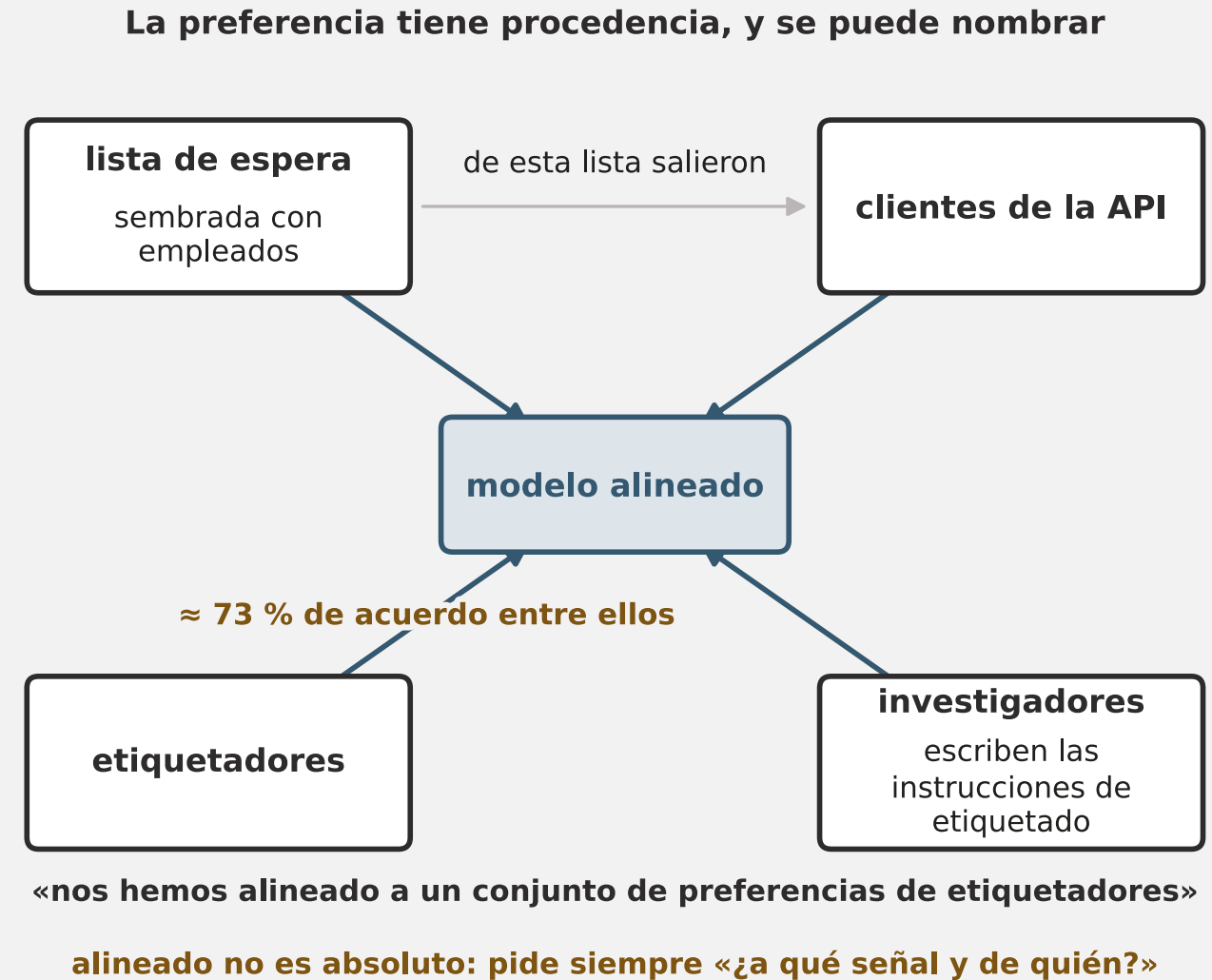
RM = modelo de recompensa · ref. = política de referencia
Bai (2022) no menciona política de referencia ni penalización KL

RLHF incluye SFT · DPO parte de él cuando existe
Constitutional AI parte de un modelo RLHF · es una pila, no un menú

Fuente: Síntesis con cada celda respaldada: Ouyang et al. (2022), §3.1, p. 6 y §3.5, pp. 8–9; Rafailov et al. (2023), §4, pp. 4–5; Bai et al. (2022), §1.2, p. 5, §3, p. 7 y §4.1, p. 11; Raschka (2024), §7.1–7.2, pp. 205–209; Alammar & Grootendorst (2024), cap. 12, §«The Three LLM Training Steps», pp. 355–357 (los tres pasos y el nombre «ajuste por preferencias»)

Alineado, ¿a qué señal y de quién?

- El paper responde en una sección entera: ‘nos hemos alineado a un conjunto de preferencias de etiquetadores’
- Cuatro procedencias, que el paper enumera por separado: los etiquetadores; los investigadores, que escriben las instrucciones; los clientes de la API; la lista de espera
- Los etiquetadores discrepan entre sí en muchos ejemplos: alrededor del 73 % de acuerdo
- ‘Es imposible que se pueda entrenar un sistema alineado a las preferencias de todos a la vez’
- Del marco útil, honesto e inofensivo, la honestidad no se mide: ‘como el modelo es una gran caja negra, no podemos inferir sus creencias’
- Se mide veracidad como proxy declarado — alucinación en dominio cerrado y TruthfulQA— y ‘solo captura una pequeña parte’



Dónde queda $P(X)$ después del alineamiento

- Preentrenar es MLE sobre $P(x_1, \dots, x_n)$ factorizado autorregresivamente: la identidad es exacta y la red aprende cada factor
- El SFT sigue siendo MLE. No cambia el objetivo: cambia el corpus, que ahora tiene forma de instrucción respondida
- RLHF y DPO ya no son MLE sobre texto: optimizan una preferencia relativa entre respuestas al mismo prompt
- Con una restricción que impide alejarse de π_{ref} . Es un objetivo distinto, no una versión más del mismo
- Lo que no cambia: en inferencia el modelo sigue produciendo una distribución sobre el vocabulario en cada paso
- El modelo alineado sigue siendo un modelo generativo autorregresivo. Lo que cambió es a qué distribución se lo empujó

